COMPUTER ENGINEERING

QUEZON CITY

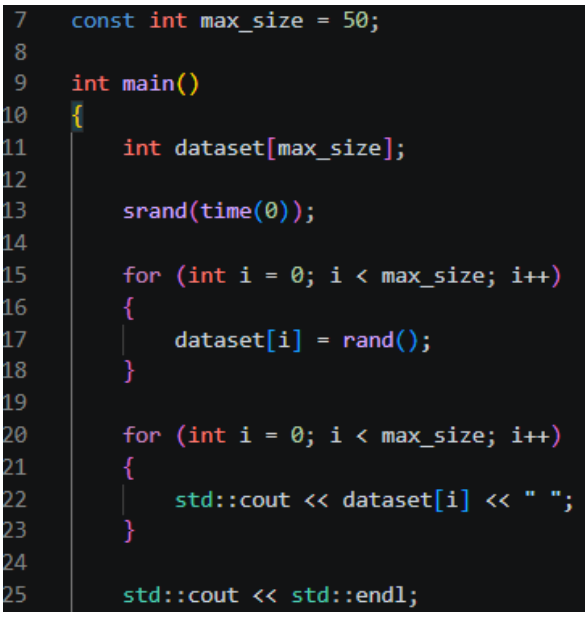
Bvs

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S1
DATE PERFORMED:	18/8/2026	**DATE SUBMITTED:**	29/8/26
NAME:	Duenas Ralph Florenz F.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

What procedures did you perform?

Screenshot - Table 6-1. Data Generated and Observations.	
Observation	It creates 50 random numbers using rand(). By using srand(time(0)), it uses your computer's clock to make sure you get a new set of numbers every time you run it. Finally, a loop prints all 50 numbers side-by-side with spaces, showing that the numbers are ready to be used.

Code - Table 6-2a. Linear Search for Arrays	`searchingalgo.h` `main.cpp`
Output	
Observations	The code searched through the list one by one for the number 999. Because 999 wasn't in the list, it checked every single number from start to finish before concluding: "The element is not found."

```cpp
#include "node.h"
#include <iostream>

#ifndef SEARCHINGALGO_H
#define SEARCHINGALGO_H

template <typename T>
void linearSearch(T data[], int arrSize, T dataFind)
{
    // Step 1: i = 0
    int i = 0;

    // Step 2: Repeat while i < n
    while (i < arrSize)
    {
        // Step 3: If item == data[i]
        if (dataFind == data[i])
        {
            std::cout << "Searching is successful" << std::endl;
            return;
        }

        // Step 4: i = i + 1
        i++;
    }

    // Step 5: If i == n
    std::cout << "The element is not found" << std:endl;
}
```

```cpp
const int max_size = 50;

int main()
{
    int dataset[max_size];

    srand(time(0));

    for (int i = 0; i < max_size; i++)
    {
        dataset[i] = rand();
    }

    for (int i = 0; i < max_size; i++)
    {
        std::cout << dataset[i] << " ";
    }

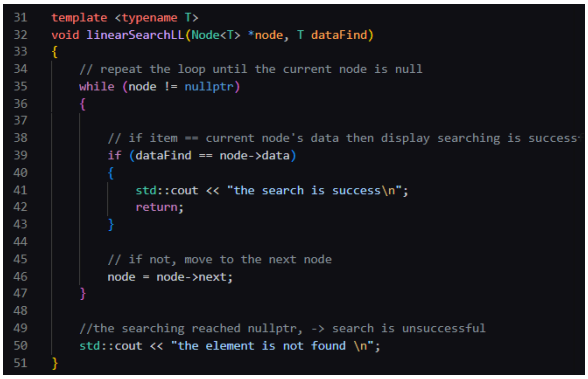
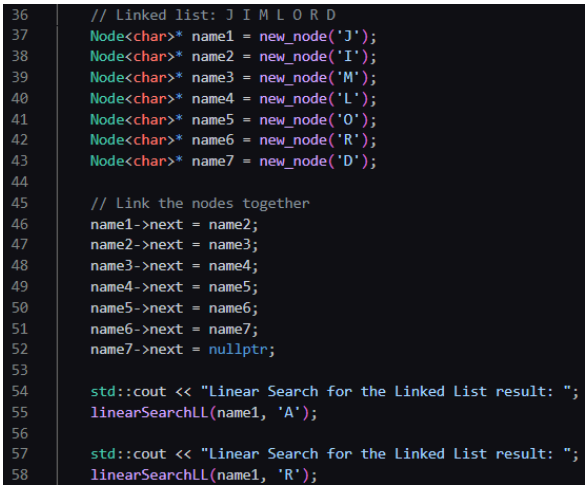
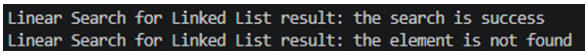
    std::cout << std::endl;

    int dataFind;

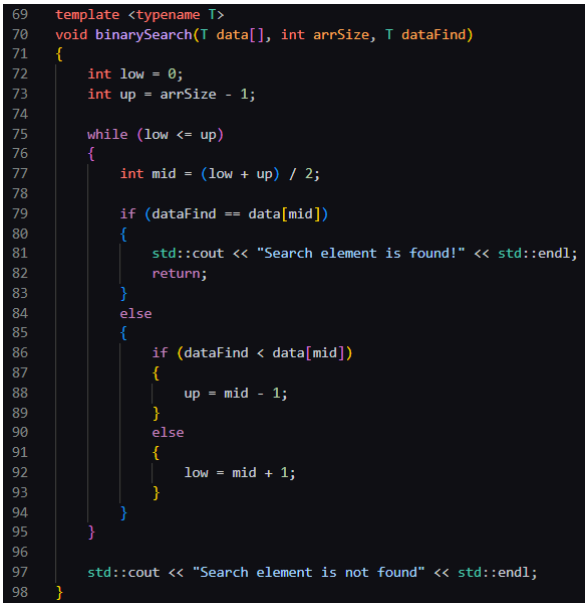
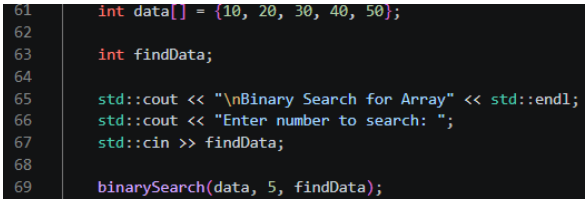
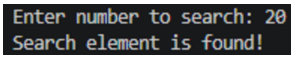
    std::cout << "Enter number to search: ";
    std::cin >> dataFind;

    linearSearch(dataset, max_size, dataFind);
```

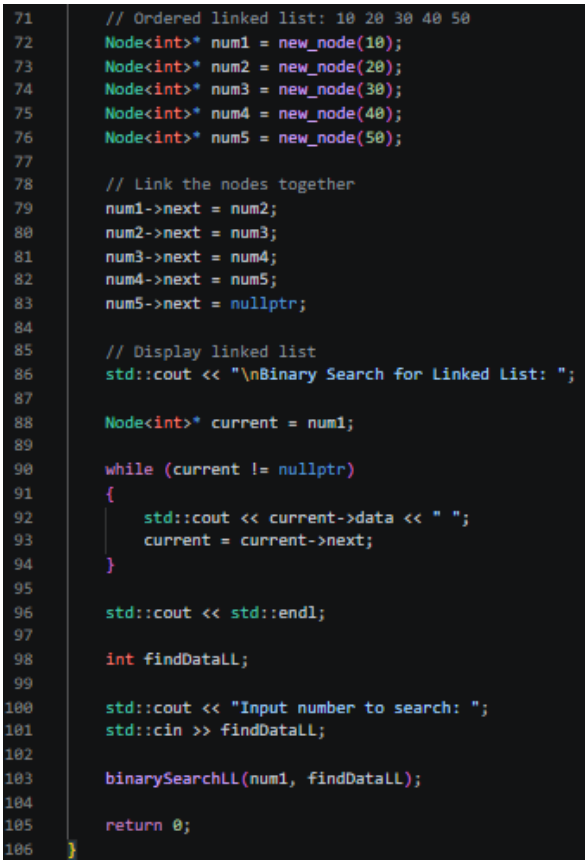
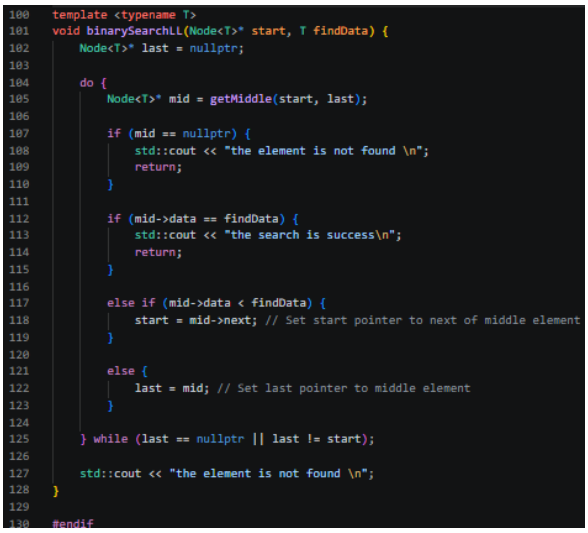
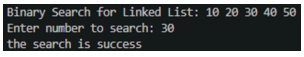
```
7686 16781 12499 23132 10467 2139 17271 5513 9553 29554 8788 21708 7995 31963 17889 15119 18672
687 24398 30457 7493 1388 4867 21130 27582 9906 30807 31064
Enter number to search: 999
The element is not found
```

Code - Table 6-2b. Linear Search for Linked-list	 
Output	
Observations	The code builds a chain of letters with J, I, M, L, O, R, and D, then searches through them one by one starting from the beginning. Searching for 'D' successfully finds the letter at the end of the list, while searching for 'A' checks every single letter without finding a match and correctly reports that it is missing.

```
Linear Search for Linked List result: the search is success
Linear Search for Linked List result: the element is not found
```

Code - Table 6-3a. Binary Search for Arrays	 
Output	
Observations	The binary search quickly found the number 20 by continually splitting the sorted list in half and checking the middle number until it reached a match.

```
Enter number to search: 20
Search element is found!
```

Code - Table 6-3b. Binary Search for Linked List	 
Output	
Observations	The binary search quickly found the value 30 in the ordered list by repeatedly checking the middle node and narrowing down the search area until it hit a match.

```
    // Ordered linked list: 10 20 30 40 50
    Node<int>* num1 = new_node(10);
    Node<int>* num2 = new_node(20);
    Node<int>* num3 = new_node(30);
    Node<int>* num4 = new_node(40);
    Node<int>* num5 = new_node(50);

    // Link the nodes together
    num1->next = num2;
    num2->next = num3;
    num3->next = num4;
    num4->next = num5;
    num5->next = nullptr;

    // Display linked list
    std::cout << "\nBinary Search for Linked List: ";

    Node<int>* current = num1;

    while (current != nullptr)
    {
        std::cout << current->data << " ";
        current = current->next;
    }

    std::cout << std::endl;

    int findDataLL;

    std::cout << "Input number to search: ";
    std::cin >> findDataLL;

    binarySearchLL(num1, findDataLL);

    return 0;
}
```

```
template <typename T>
void binarySearchLL(Node<T>* start, T findData) {
    Node<T>* last = nullptr;

    do {
        Node<T>* mid = getMiddle(start, last);

        if (mid == nullptr) {
            std::cout << "the element is not found \n";
            return;
        }

        if (mid->data == findData) {
            std::cout << "the search is success\n";
            return;
        }

        else if (mid->data < findData) {
            start = mid->next; // Set start pointer to next of middle element
        }

        else {
            last = mid; // Set last pointer to middle element
        }

    } while (last == nullptr || last != start);

    std::cout << "the element is not found \n";
}

#endif
```

```
Binary Search for Linked List: 10 20 30 40 50
Enter number to search: 30
the search is success
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILOB

- Problem 1. Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons would be necessary to find the key '18'?

```cpp
#include <iostream>

int main()
{
    int data[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int key = 18;
    int comparisons = 0;

    for (int i = 0; i < 10; i++)
    {
        comparisons++;

        if (data[i] == key)
        {
            std::cout << "Found: " << data[i] << std::endl;
            break;
        }
    }

    std::cout << "Comparisons: " << comparisons << std::endl;

    return 0;
}
```

```cpp
#include <iostream>
#include "node.h"

int main()
{
    Node<int>* n1 = new_node(15);
    Node<int>* n2 = new_node(18);
    Node<int>* n3 = new_node(2);
    Node<int>* n4 = new_node(19);
    Node<int>* n5 = new_node(18);
    Node<int>* n6 = new_node(0);
    Node<int>* n7 = new_node(8);
    Node<int>* n8 = new_node(14);
    Node<int>* n9 = new_node(19);
    Node<int>* n10 = new_node(14);

    n1->next = n2;
    n2->next = n3;
    n3->next = n4;
    n4->next = n5;
    n5->next = n6;
    n6->next = n7;
    n7->next = n8;
    n8->next = n9;
    n9->next = n10;
    n10->next = nullptr;

    Node<int>* current = n1;
    int key = 18;
    int comparisons = 0;

    while (current != nullptr)
    {
        comparisons++;

        if (current->data == key)
        {
            std::cout << "Found: " << current->data << std::endl;
            break;
        }

        current = current->next;
    }

    std::cout << "Comparisons: " << comparisons << std::endl;

    return 0;
}
```

```
Found: 18
Comparisons: 2
```

- Problem 2. Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element ‘k’. Test on the same list given in problem 1.

```cpp
#include <iostream>

int countRepeating(int data[], int size, int key)
{
    int count = 0;

    for (int i = 0; i < size; i++)
    {
        if (data[i] == key)
        {
            count++;
        }
    }

    return count;
}

int main()
{
    int data[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};

    int key = 18;

    int result = countRepeating(data, 10, key);

    std::cout << "Total number of occurences: "
              << key << ": " << result << std::endl;

    return 0;
}
```

```
Number of occurrences of 18: 2
```

- Problem 3. Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you wanted to find the key 8, draw a diagram that shows how the searching works per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a screenshot of the code and the output console.

```cpp
#include <iostream>

int main()
{
    int data[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};

    int key = 8;
    int low = 0;
    int up = 9;
    int comparisons = 0;

    while (low <= up)
    {
        int mid = (low + up) / 2;

        comparisons++;

        std::cout << "Iteration " << comparisons
                  << ": mid = " << data[mid] << std::endl;

        if (data[mid] == key)
        {
            std::cout << "Search element is found!" << std::endl;
            std::cout << "Index: " << mid << std::endl;
            std::cout << "Comparisons: " << comparisons << std::endl;
            return 0;
        }
        else if (key < data[mid])
        {
            up = mid - 1;
        }
        else
        {
            low = mid + 1;
        }
    }

    std::cout << "No search element is found " << std:: endl;

    return 0;
}
```

```
Iteration 1: mid = 11
Iteration 2: mid = 5
Iteration 3: mid = 6
Iteration 4: mid = 8
Search element is found!
Index: 3
Comparisons: 4
```

- Problem 4. Modify the binary search algorithm so that the algorithm becomes recursive. Using this new recursive binary search, implement a solution to the same problem for problem 3.

```cpp
#include <iostream>

int recursiveBinarySearch(int data[], int low, int up, int key)
{
    if (low > up)
    {
        return -1;
    }

    int mid = (low + up) / 2;

    std::cout << "Checking: " << data[mid] << std::endl;

    if (data[mid] == key)
    {
        return mid;
    }

    if (key < data[mid])
    {
        return recursiveBinarySearch(data, low, mid - 1, key);
    }

    return recursiveBinarySearch(data, mid + 1, up, key);
}

int main()
{
    int data[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};

    int key = 8;

    int result = recursiveBinarySearch(data, 0, 9, key);

    if (result != -1)
    {
        std::cout << "Search element is found!" << std::endl;
        std::cout << "Index: " << result << std::endl;
    }
    else
    {
        std::cout << "No search element is found " << std:: endl;
    }

    return 0;
}
```

```
Checking: 11
Checking: 5
Checking: 6
Checking: 8
Search element is found!
Index: 3
```

- **Analysis of the supplementary activity**

In the supplementary activity, the program demonstrates various array and linked list operations along with sequential and binary search algorithms. It first calculates the number of comparisons required to find a target key in an unsorted list using both an array and a singly linked list. The program then modifies the sequential search function to iterate through the entire array and count all repeating occurrences of a specific element. Next, it applies an iterative binary search to a sorted array to efficiently locate a key while tracking the midpoint values and comparison count for each step. Finally, the program converts the binary search into a recursive function that repeatedly divides the search space in half until it finds the target element and returns its index.

- **Concluding statement / Feedback:**

I think I had a little bit of a hard time in this HOA because I was short on time during the exam week but I still had the ideas from the past lesson. Yet i was able to successfully implement both sequential and binary search algorithms across arrays and singly linked lists while accurately tracking comparison counts and repeating instances. However, I still need more practice in converting iterative algorithms into clean recursive functions and ensuring proper base cases to prevent unintended stack overflow errors during execution.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Copilot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code Assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used Copilot to elaborate the thoughts I have in my mind without replacing the original thought and also for getting the formulas that are needed. I affirm that my use of AI complies with TIP's AI usage policy.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.